

Dokumentation – Eventure

Raka Pradnya Putra Adita - 8778662

Dieses Projekt ist auf Github unter dem folgenden Link zu finden:

https://github.com/rakaputra12/Software_Engineering_Assignment

Zielsetzung der Anwendung

Die Anwendung wurde entwickelt, um den Nutzern eine einfache und intuitive Möglichkeit zu bieten, Events in ihrer Umgebung zu suchen, zu verwalten und zu speichern. Mit einem benutzerfreundlichen Interface soll das Erlebnis der Eventplanung optimiert werden. Die Hauptziele sind:

1. **Effiziente Suche nach Events:** Nutzer können durch Eingabe von Stadt, Kategorie und Suchradius gezielt Events finden, die ihren Interessen entsprechen.
2. **Dynamische Kartendarstellung:** Mit der Integration von Kartenvisualisierungen können Nutzer die geografische Lage der Events sehen und schnell relevante Informationen abrufen.
3. **Bookmark-Funktionalität:** Lieblings-Events können gespeichert werden, sodass Nutzer ihre favorisierten Veranstaltungen leicht wiederfinden.

Funktionalität der Anwendung

Die Anwendung bietet folgende Kernfunktionen:

1. **Eventsuche:**
 - Nutzer können eine Stadt, eine Kategorie (z. B. Musik, Sport, Film) und einen Suchradius angeben, um relevante Events in der Umgebung zu finden.
 - Ergebnisse werden in einer Liste angezeigt, die Details wie den Namen des Events, Veranstaltungsort und Datum enthält.
2. **Kartendarstellung:**
 - Die gefundenen Events werden auf einer interaktiven Karte angezeigt.
 - Marker auf der Karte zeigen den genauen Veranstaltungsort an, und Nutzer können mit einem Klick weitere Details abrufen.
3. **Bookmark-Funktion:**
 - Nutzer können Events als Favoriten speichern, um sie später erneut aufzurufen.
 - Favoriten werden in einem separaten Bereich angezeigt, der leicht zugänglich ist.
4. **Responsives Design:**

- Die Anwendung wurde mit einem responsiven UI entwickelt, das auf verschiedenen Bildschirmgrößen gut funktioniert.
 - Die Benutzeroberfläche ist intuitiv und einfach zu navigieren.
- 5. Backend-Integration:**
- Das Backend nutzt eine RESTful API, die die Daten der Events bereitstellt und alle Benutzeranfragen verarbeitet.
 - Daten werden aus externen APIs (z. B. Ticketmaster) abgerufen und verarbeitet.

Anforderungen der Anwendung

Muss-Anforderungen

- 1. Eventsuche:**
 - Die Anwendung muss eine Suchfunktion bereitstellen, bei der Nutzer eine Stadt, Kategorie und einen Suchradius angeben können.
 - Die Ergebnisse müssen in einer Liste angezeigt werden, die die wichtigsten Informationen zu jedem Event enthält.
- 2. Kartenintegration:**
 - Die Anwendung muss eine dynamische Karte integrieren, die die gefundenen Events mit Markern anzeigt.
 - Die Karte muss durch Klicken auf einen Marker zusätzliche Informationen zu einem Event bereitstellen.
- 3. Bookmark-Funktion:**
 - Nutzer müssen in der Lage sein, Events zu favorisieren und diese in einer separaten Liste zu speichern.
 - Favoriten müssen zwischen den Sitzungen durch Speicherung im Session Storage erhalten bleiben.
- 4. Backend-Anbindung:**
 - Die Anwendung muss ein Backend nutzen, das Anfragen von der Benutzeroberfläche verarbeitet und Daten von einer externen API (z. B. Ticketmaster) abrufen.
- 5. Responsives Design:**
 - Die Benutzeroberfläche muss auf verschiedenen Geräten, einschließlich Desktop und Mobilgeräten, funktionsfähig und benutzerfreundlich sein.

Soll-Anforderungen

- 1. Filtern nach Kategorien:**
 - Nutzer sollen nach spezifischen Kategorien wie "Musik", "Sport" oder "Film" filtern können, um die Suche zu verfeinern.
- 2. Fehlerbehandlung:**
 - Falls keine Events gefunden werden, soll eine benutzerfreundliche Fehlermeldung angezeigt werden.
- 3. Interaktive Benutzerführung:**

- Die Anwendung soll visuelle und textuelle Hinweise bieten, um die Nutzer durch den Suchprozess zu führen.

Kann-Anforderungen

1. Pagination der Ergebnisse:

- Die Eventliste kann eine Paginierung implementieren, um große Datenmengen übersichtlich darzustellen.

2. Zusätzliche Kartenfunktionen:

- Es kann eine Funktion integriert werden, um von der Kartenansicht direkt zur Detailansicht eines Events zu gelangen.

3. Benachrichtigungsfunktion:

- Nutzer können optional Benachrichtigungen erhalten, wenn neue Events in ihrer bevorzugten Kategorie verfügbar sind.

Die umgesetzte Anforderungen

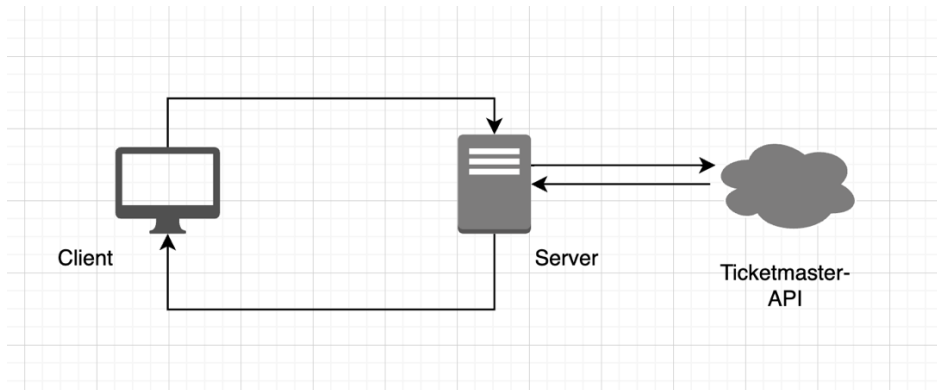
Anforderung	Implementiert	Nicht Implementiert
Eventsuche	x	
Kartenintegration	x	
Bookmark-Funktion	x	
Backend-Anbindung	x	
Responsives Design	x	
Filtern nach Kategorien	x	
Fehlerbehandlung	x	
Interaktive Benutzerführung	x	
Pagination der Ergebnisse	x	
Benachrichtigungsfunktion		x
Zusätzliche Kartenfunktionen		x

Architekturmuster

Die Anwendung folgt dem **Client-Server-Architekturmuster**, bei dem das Frontend und das Backend klar voneinander getrennt sind:

- **Frontend:** Implementiert in React.js mit **Vite** als Build-Tool. Das Frontend stellt die Benutzeroberfläche bereit, ermöglicht die Eingabe von Suchparametern, zeigt Ergebnisse und Favoriten an und integriert eine interaktive Karte.
- **Backend:** Implementiert in FastAPI (Python), das die Verarbeitung von Anfragen übernimmt und Daten von der **Ticketmaster-API** abrufen. Das Backend dient als Middleware zwischen der externen API und dem Frontend.

- **Datenhaltung:** Die Anwendung nutzt den **Session Storage** des Browsers zur Speicherung von Favoriten, um Daten über die Sitzungsdauer hinweg zu bewahren.



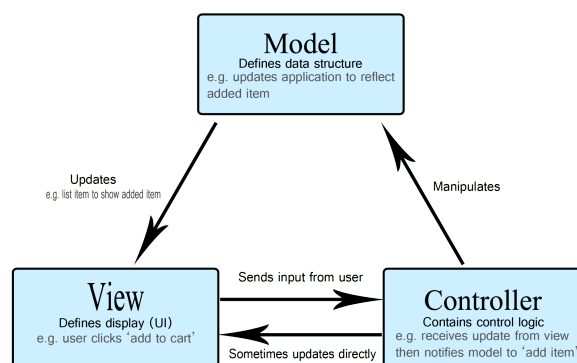
Dieses Muster bietet eine klare Trennung der Verantwortlichkeiten, verbessert die Skalierbarkeit und erleichtert die Wartung.

Design Pattern

In der Anwendung wurden folgende Design Patterns umgesetzt:

1. Model-View-Controller (MVC):

- **Model** Das Backend (FastAPI) übernimmt die Datenverarbeitung und stellt die Datenmodelle sowie API-Endpunkte für das Frontend bereit.
- **View:** Das Frontend (React) zeigt die Daten in Form von Listen, Karten und Favoritenlisten an.
- **Controller:** Die Logik zur Verarbeitung von Benutzereingaben und API-Anfragen wird sowohl im Backend (FastAPI) als auch im Frontend (React) implementiert.



2. Observer-Pattern:

- Im Frontend wird der Zustand der Anwendung (z. B. Favoriten) mithilfe von `useState` und `useEffect` verwaltet. Änderungen an diesem

Zustand führen zu einer automatischen Aktualisierung der Benutzeroberfläche.

3. Factory Pattern:

- Die Erstellung von Kartenmarkern (Leaflet-Icons) basiert auf einer Factory-Methode, die Marker mit verschiedenen Icons für ausgewählte und nicht ausgewählte Events erstellt. Dadurch kann das Icon dynamisch je nach Event-Status angepasst werden.

4. Singleton Pattern:

- Im Backend gibt es eine einzige Instanz der FastAPI-App.

```
from fastapi import FastAPI
app = FastAPI()
```

- Diese Singleton-Instanz wird für alle Anfragen wiederverwendet, um Effizienz zu gewährleisten.

5. Decorator Pattern:

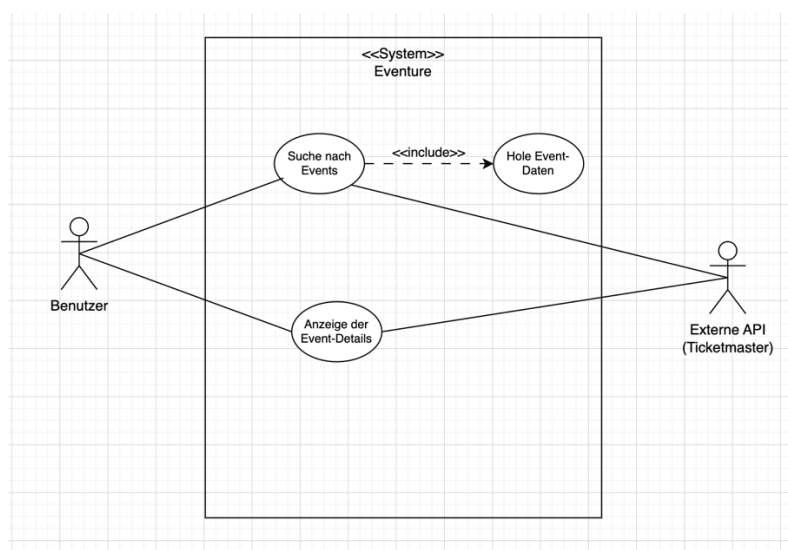
- FastAPI nutzt das Decorator-Pattern für API-Endpunkte.

```
@app.get("/api/events")
def get_events():
    return {"message": "Event list"}
```

- `@app.get("/api/events")` erweitert die Funktionalität, indem er die Funktion `get_events()` als API-Route registriert.

Use Case Diagramm

Das folgende Use-Case-Diagramm zeigt in kompakter Form die Interaktion zwischen dem Benutzer, dem System „Eventure“ und der externen Ticketmaster-API. Dabei dient der Use Case „Suche nach Events“ als zentraler Vorgang, der über die „<<include>>“-Beziehung den Datenabruf („Hole Event-Daten“) von der API auslöst. Zusätzlich kann der Benutzer über den Use Case „Anzeige der Event-Details“ weitere Informationen zu einzelnen Events einsehen.



Verwendete Services und Technologien

Backend-Services

1. Ticketmaster-API:

- Wird für die Suche und Abruf von Eventdaten (z. B. Name, Veranstaltungsort, Zeit, Kategorie) verwendet.
- Anbindung erfolgt über das FastAPI-Backend, das die API-Anfragen verarbeitet und Ergebnisse an das Frontend weiterleitet.

2. FastAPI:

- Dient als Backend-Framework für die Verarbeitung von HTTP-Anfragen und die Integration der Ticketmaster-API.
- Unterstützt eine schnelle und effiziente Entwicklung mit minimalem Overhead.

Frontend-Technologien

1. React.js mit Vite:

- **React.js** ist das Hauptframework für die Entwicklung des Frontends. Es bietet eine komponentenbasierte Struktur, die die Wartbarkeit und Wiederverwendbarkeit des Codes verbessert.
- **Vite** wird als Build-Tool und Entwicklungsserver verwendet. Es ermöglicht schnelle Entwicklungszyklen durch Hot Module Replacement (HMR) und optimierte Build-Zeiten.

2. TailwindCSS:

- Verwendet für die Gestaltung der Benutzeroberfläche. Es ermöglicht die schnelle Erstellung eines responsiven Designs durch Utility-Klassen.

3. React-Leaflet:

- Wird für die Kartenintegration verwendet. Diese Bibliothek nutzt die Funktionen von Leaflet, um Marker und Popups für Events auf einer dynamischen Karte darzustellen.

4. Session Storage:

- Wird genutzt, um die Favoriten der Benutzer lokal im Browser zu speichern. Dies ermöglicht eine benutzerfreundliche Erfahrung, ohne ein komplexes Datenbanksystem implementieren zu müssen.

Anleitung zum Starten der Anwendung

Voraussetzungen

1. Installierte Software:

- **Node.js** (empfohlen: Version 16 oder höher) für das Frontend.
- **Python** (empfohlen: Version 3.10 oder höher) für das Backend.

2. Paketmanager:

- *npm* (kommt mit Node.js) für die Abhängigkeiten des Frontends.
- *pip* für die Abhängigkeiten des Backends.

3. Abhängigkeiten installieren:

- Stellen Sie sicher, dass *npm* und *pip* verfügbar sind und korrekt installiert wurden.

4. Zusätzliche Tools:

- **Vite** (enthalten in den Projektabhängigkeiten).
- Ein Browser wie **Google Chrome**.

Schritt 1: Backend Starten (FastAPI)

1. Navigieren Sie in das server-Verzeichnis:

```
cd server
```

2. Erstellen Sie eine virtuelle Umgebung und aktivieren Sie sie:

```
python -m venv .venv  
  
source .venv/bin/activate    # Für macOS/Linux  
.venv\Scripts\activate      # Für Windows
```

3. Installieren Sie die Python-Abhängigkeiten:

```
pip install -r requirements.txt
```

4. Starten Sie den FastAPI-Server:

```
uvicorn app:app --reload
```

- Der Server sollte standardmäßig auf *http://127.0.0.1:8000* verfügbar sein.

Schritt 2: Frontend starten (Vite + React.Js)

1. Öffnen Sie ein neues Terminal und navigieren Sie in das client-Verzeichnis:

```
cd client
```

2. Installieren Sie die JavaScript-Abhängigkeiten:

```
npm install
```

3. Starten Sie die Entwicklungsumgebung:

```
npm run dev
```

- Das Frontend sollte standardmäßig auf `http://localhost:5173` verfügbar sein.

Schritt 3: Anwendung verwenden

1. Öffnen Sie Ihren Browser und navigieren Sie zu `http://localhost:5173`.
2. Geben Sie die gewünschte Stadt, Kategorie und den Radius in das Suchformular ein.
3. Überprüfen Sie die angezeigten Veranstaltungen, verwenden Sie die Karte und fügen Sie Ereignisse zu den Favoriten hinzu.

Hinweis

- Stellen Sie sicher, dass sowohl das Backend (FastAPI) als auch das Frontend (Vite) parallel ausgeführt werden.
- Falls Fehler auftreten:
 - Überprüfen Sie die installierten Abhängigkeiten (npm install für das Frontend, pip install für das Backend).
 - Prüfen Sie die Konsolenausgabe von uvicorn und npm run dev auf Fehlermeldungen.

The screenshot shows the Eventure web application interface. It features a central search form with the following fields: a text input for 'City' with a placeholder 'Enter a city', a dropdown menu for 'Category' with the text 'Select a category', and a text input for 'Radius (km)' with a placeholder 'e.g., 10'. Below these fields is a blue button labeled 'Find Events'. To the left of the form is a section titled 'Discover Events Near You' with the text 'Find the best concerts, sports games, and theater shows in your area with just a few clicks!'. To the right is a section titled 'Save Your Favorites' with the text 'Bookmark events you love and never miss out on the action. Your personalized event planner is here.'

City
Berlin

Category
Music

Radius (km)
10

Showing 1 event(s) within 10 km

Date: February 13, 2025 at 19:30

[View Details](#)

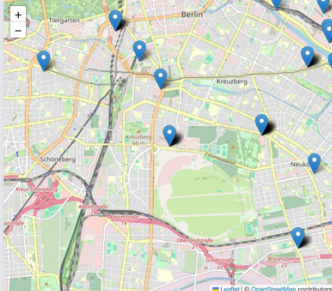


Odetari - VARIA 0.5 EU TOUR

Venue: Holzmarktstraße 25

Date: February 13, 2025 at 20:00

[Unbookmark](#)



Bookmarked Events

Anleitung zum Ausführen der Tests

Voraussetzungen

1. Backend:

- Stellen Sie sicher, dass das Backend gestartet ist.
- Wechseln Sie in das Verzeichnis server und starten Sie den Server:

```
cd server
uvicorn app:app --reload
```

Der Server sollte auf *http://127.0.0.1:8000* laufen.

2. Frontend:

- Stellen Sie sicher, dass das Frontend gestartet ist.
- Wechseln Sie in das Verzeichnis client und starten Sie das Frontend:

```
cd client
npm run dev
```

Das Frontend sollte auf *http://localhost:5173* verfügbar sein.

3. Zusätzliche Anforderungen:

- **Selenium:**
 - Installieren Sie selenium für Python:
- ```
pip install selenium
```
- Stellen Sie sicher, dass **ChromeDriver** installiert und in Ihrem PATH ist.
  - Laden Sie die richtige Version von ChromeDriver herunter, die zu Ihrer Google Chrome-Version passt.
  - **Unittest:**
    - Python enthält unittest standardmäßig. Keine zusätzliche Installation ist erforderlich.

### Schritt 1: Unit-Tests ausführen

1. Navigieren Sie in das Hauptverzeichnis des Projekts:

```
cd /Software_Engineering_Assignment
```

2. Aktivieren Sie die virtuelle Umgebung:

```
source server/.venv/bin/activate
```

3. Führen Sie die Test im Verzeichnis tests mit folgendem Befehl aus:

```
python3 -m unittest discover -s tests
```

#### 4. Erwartetes Ergebnis:

- Die Konsole zeigt Testergebnisse (z. B. Passed/Failed) mit Details zu allen Tests.

### Schritt 2: Selenium-Tests ausführen

1. Navigieren Sie in das Verzeichnis tests:

```
cd /Software_Engineering_Assignment/tests
```

2. Stellen Sie sicher, dass die virtuelle Umgebung aktiviert.
3. Starten Sie den Selenium-Test:

```
python3 selenium_test.py
```

#### 4. Erwartetes Ergebnis:

- Der Selenium-Test öffnet einen Chrome-Browser.
- Die Anwendung wird im Browser geladen, und der Test interagiert automatisch mit der Benutzeroberfläche (z. B. Eingabe der Stadt, Kategorieauswahl, Bookmarking von Events).
- Nach Abschluss des Tests wird ein Ergebnis in der Konsole ausgegeben:
  - "Test Passed: Event was successfully bookmarked!" bei Erfolg.
  - "Test Failed: [Fehlerdetails]" bei einem Fehler.

#### 5. Fehlerbehebung:

- Überprüfen Sie, ob der ChromeDriver korrekt installiert ist und mit Ihrer Chrome-Version übereinstimmt.
- Stellen Sie sicher, dass das Backend und das Frontend laufen, bevor Sie den Test starten.

---

### Test-Details

#### 1. Unit-Tests (test\_unit.py):

- Testet die API des Backends.
- Überprüft die korrekte Verarbeitung von Event-Daten und die Rückgabe der Ergebnisse.

#### 2. Selenium-Tests (selenium\_test.py):

- Automatisiert die Benutzeroberfläche.
- Überprüft die Funktionalität der Anwendung, einschließlich Eingabe, Suche und Bookmarking von Events.